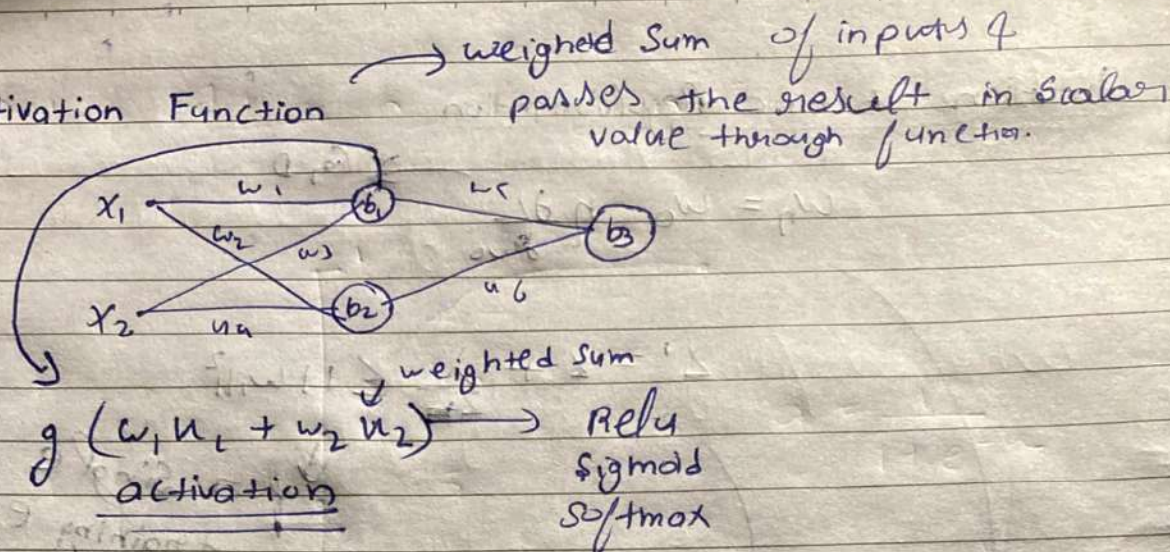


Activation Function



Agar Ap activation function Apne neural network mein Apply hhi krnge toh vo linear data sirf capture krrega, non linear data capture nhi kr payega

Ideal Activation function

1) Non linear

↳ if data have non linear pattern then also it can capture

2) Differentiable

↳ can calculate derivatives so that we can find gradient descent

(Relu is not diff at 0 but we still use)

3) Computationally inexpensive

↳ derivative → Simple

otherwise training will be slow

4) Zero Centered

↳ output should be ^{zero centered} normalized

↓ mean around zero ↓ convergence fast

ex → tanh

5) Non-saturating →

ReLU → $\max(0, u)$

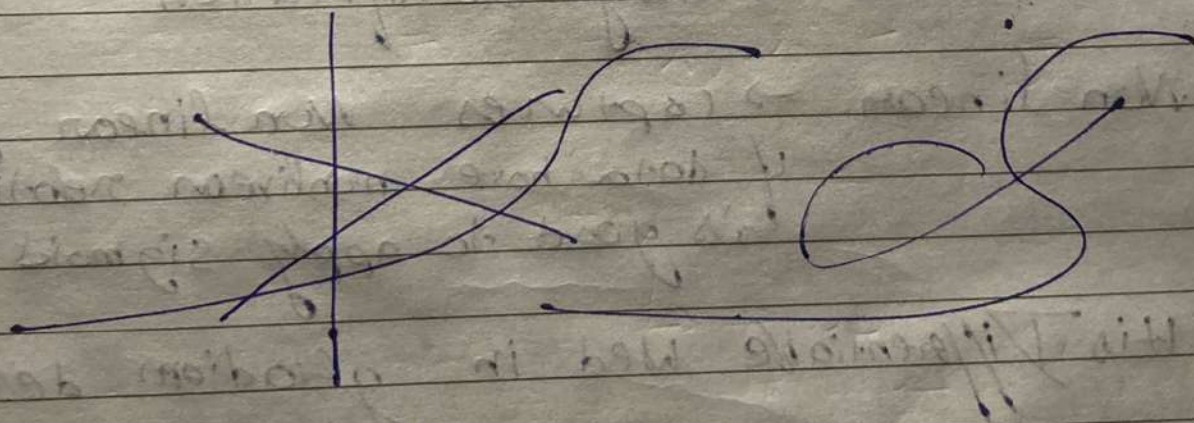
↳ non-saturating
no fix range

Sigmoid → 0, 1
tan → -1, 1

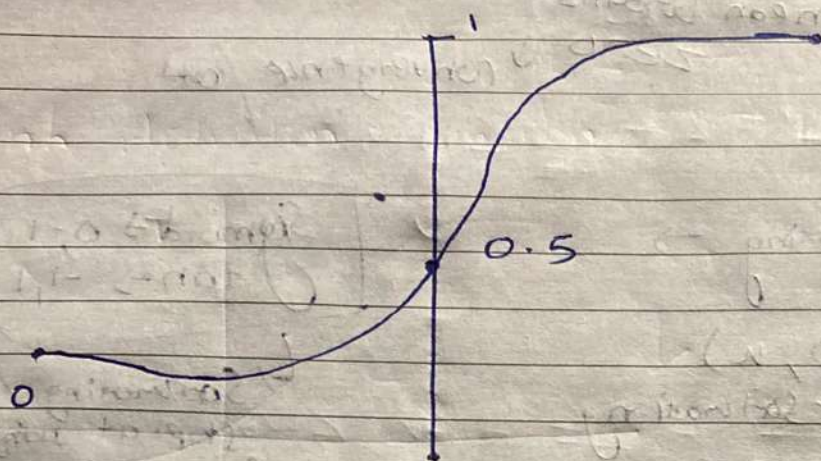
↳ saturating func
stops at range

vanishing gradient problem
occur, small × small =
very small GD
So, no update

~~Sigmoid Activation function~~



Sigmoid Activation function



$$a = \frac{1}{1 + e^{-u}}$$

if $u \uparrow$ Sigmoid ≈ 1
if $u \downarrow$ Sigmoid ≈ 0
if $u = 0$ Sigmoid ≈ 0.5

Advantage

- 1] Range $[0, 1]$ can be treated as probability
↓
Binary classification
- 2] Non linear $\rightarrow$ captures Non linear pattern,
if data have nonlinear relation it
is good to apply sigmoid
- 3] It is Differentiable used in gradient descent

Disadvantages

- 1) Saturating function $\rightarrow$ Squish given input into a range
 $\hookrightarrow$ problem of vanishing gradient problem

$$w_n = w_0 - \eta \left[\frac{\partial L}{\partial w} \right]$$

$\left[\frac{\partial L}{\partial w} \right] \approx 0$
 $\downarrow$
 $w_n = w_0$
 no update
 $\downarrow$
 So no training

why ≈ 0

Sigmoid only used in output layer for binary classification

2) Non Zero centered

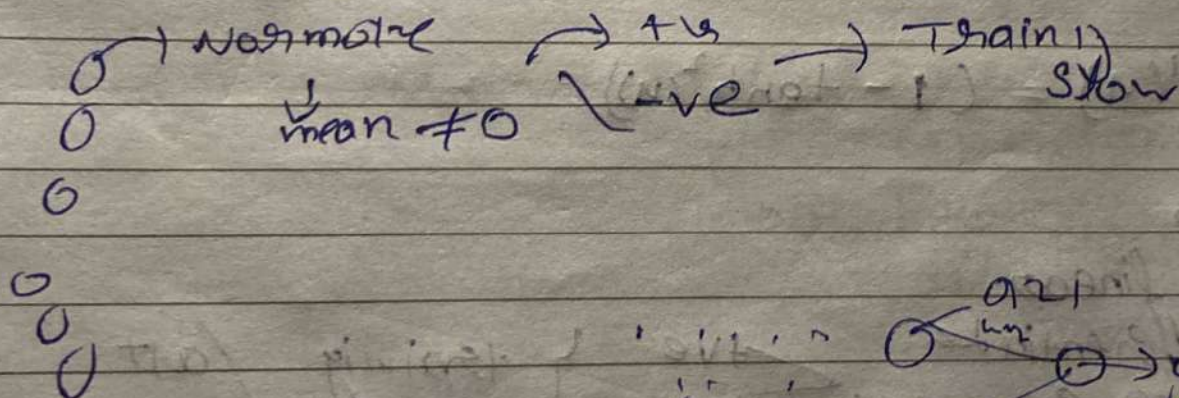


Diagram illustrating the forward pass for a single neuron in a hidden layer:

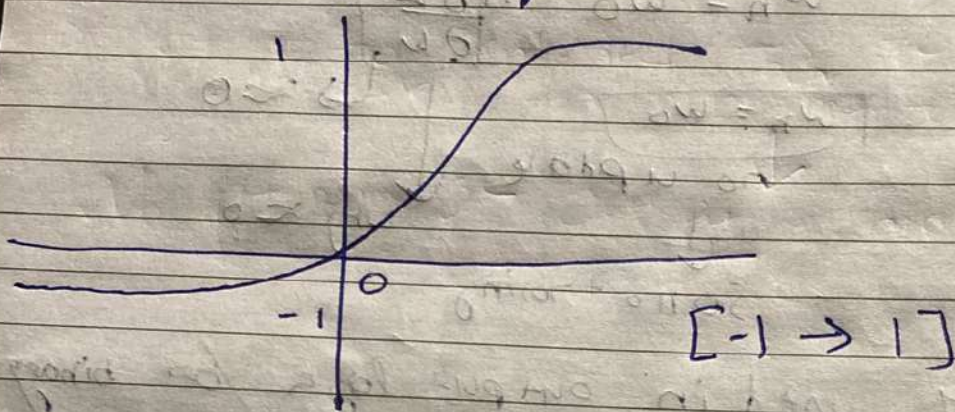
Input x is multiplied by weight w_{21} to produce z_{21} . The bias b_{21} is added to z_{21} to produce the net input a_{21} . The net input a_{21} is then passed through the sigmoid function σ to produce the output y_1 .

The backward pass (backpropagation) is shown as:

$$\frac{\partial L}{\partial w_{21}} = \left[\frac{\partial L}{\partial y_1} \times \frac{dy_1}{da_{21}} \times \frac{da_{21}}{dz_{21}} \right] \times \frac{\partial z_{21}}{\partial w_{21}}$$

3) $\sigma(z) = \frac{1}{1+e^{-z}} \rightarrow$ computational expensive because of exponent

Tanh Activation function



$$f(u) = \frac{e^u - e^{-u}}{e^u + e^{-u}}$$

$$f'(u) = (1 - \tanh^2(u))$$

Advantage

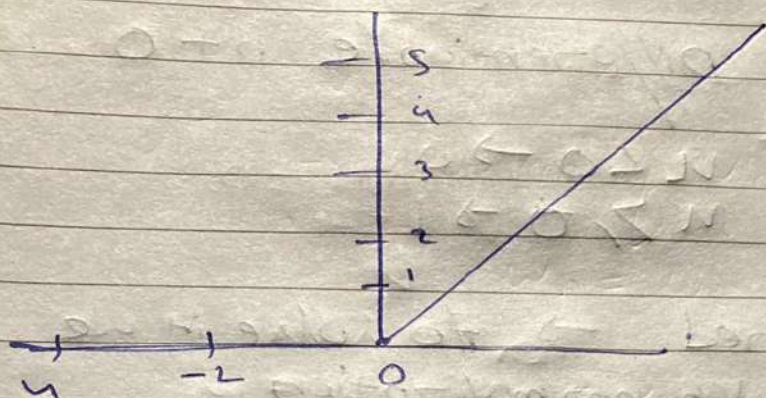
- 1) Non linear
- 2) Differentiable
- 3) zero centered $\begin{matrix} \leftarrow +ve \\ -ve \end{matrix} \} \text{ training fast}$

Disadvantages

- $\rightarrow$ Saturating function $\rightarrow$ vanishing gradient problem
- $\rightarrow$ Computational expensive due to exponent

Relu Activation Function

Best Activation function



$$f(u) = \max(0, u)$$

for -ve value is 0
for +ve value same value output

Advantages

1) Non linear

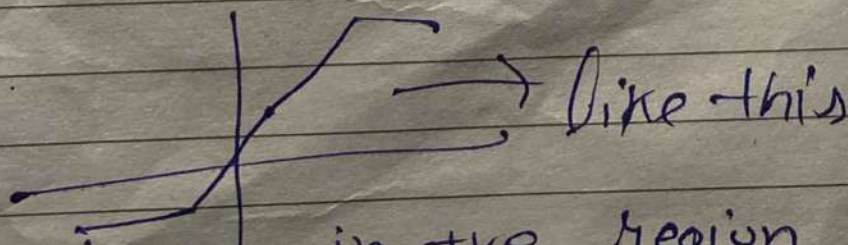
if someone ask

$$f(u) = \max(0, u)$$

↳ How it is non linear, because we can

Add two or more relu like

$$f(u) = \max(0, x+1) - \max(0, x)$$



- 2) non saturating in +ve region
- 3) computationally inexpensive $\rightarrow$ no exponent
- 4) convergence is fast than Sigmoid or tanh

Disadvantage

→ Not completely differentiable at 0

$$u < 0 \rightarrow 0$$

$$u > 0 \rightarrow 1$$

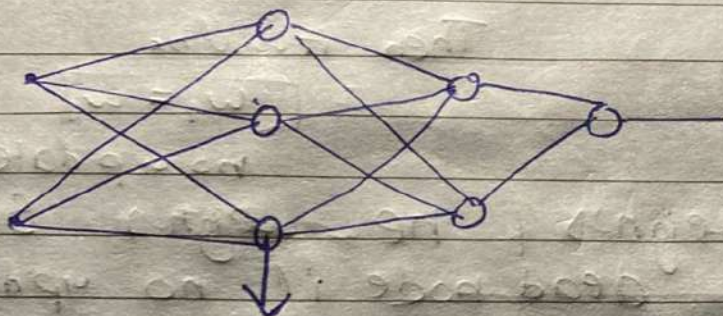
→ Not zero centered → No value it we gonna use batch normalization

↓
it normalize output of hidden layers

Dying ReLU Problem

ReLU

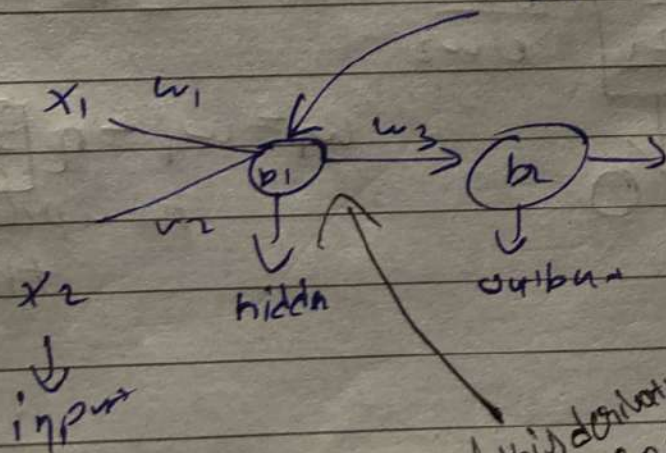
↳ Best choice for hidden layers



↳ If output is zero of any input then this is known as dead neuron, it remains forever dead

↳ If 50% > neurons are dead then model is inefficient, data pattern can't be recognized, worst case 100% neurons are dead, so no pattern recognition.

Why dying ReLU $\rightarrow$



$$a_1 = \max(0, z_1)$$

$$z_1 = w_{11} + w_{12} + b_1$$

if it's less than zero
 $a_1 = 0$

$$\frac{da_1}{dz_1} = 0$$

↳ this derivative is zero so the weights will never get updated

$$w_1 = w_1 - \eta \frac{dL}{dw_1} \Rightarrow w_1 - \eta \left(\frac{\partial L}{\partial y^1} \times \frac{dy^1}{da_1} \times \frac{da_1}{dz_1} \times \frac{dz_1}{dw_1} \right)$$

$$w_2 = w_2 - \eta \frac{dL}{dw_2} = w_1 - \eta \left(\dots \times \frac{dz_1}{dw_2} \right)$$

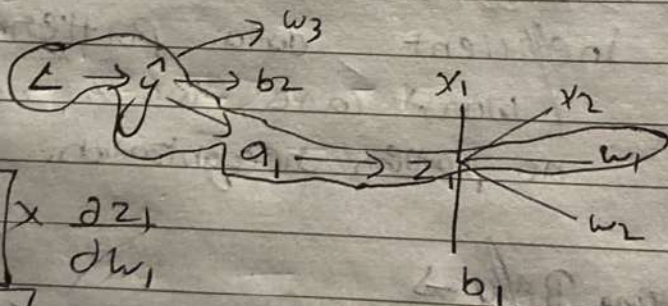
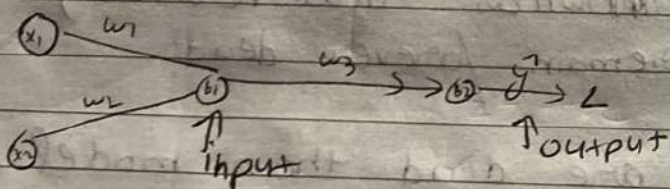
$$\frac{da_1}{dz_1} = 0$$

Then it will be

$$w_1 = w_1$$

no update

No diff in old weights & new weights So it will be dead node if no update



$$\frac{\partial L}{\partial w_1} = \frac{\partial L}{\partial y^1} \times \frac{dy^1}{da_1} \times \frac{da_1}{dz_1} \times \frac{dz_1}{dw_1}$$

$$\frac{\partial L}{\partial w_2} = \frac{\partial L}{\partial y^1} \times \frac{dy^1}{da_1} \times \frac{da_1}{dz_1} \times \frac{dz_1}{dw_2}$$

$$w_1 = w_1 - \eta \frac{\partial L}{\partial w_1}$$

no update

this is dying node problem

Causes of dying state

$$z_1 = w_1 u_1 + w_2 u_2 + b$$

$$w_1 = w_1 - \eta \frac{\partial L}{\partial w_1}$$

if it is +ve

if high LR

so new weight

will -ve

this will also +ve

so in

next cycle

z_1 may be

-ve ~~not~~

1) High learning rate (η)

2) High -ve bias

$$z_1 = w_1 u_1 + w_2 u_2 + \underline{b_1}$$



Then whole equation be -ve

$$b_1 < 0$$

become -ve

Why bias -ve

1) Initialization of bias

2) we update bias like $b_1 = b - \eta \delta$



high LR

If any time z_1 gets -ve then it will not be declared as dead, in future it will not give +ve output

$$z_1 = w_1 u_1 + w_2 u_2 + \boxed{b_1} < 0$$

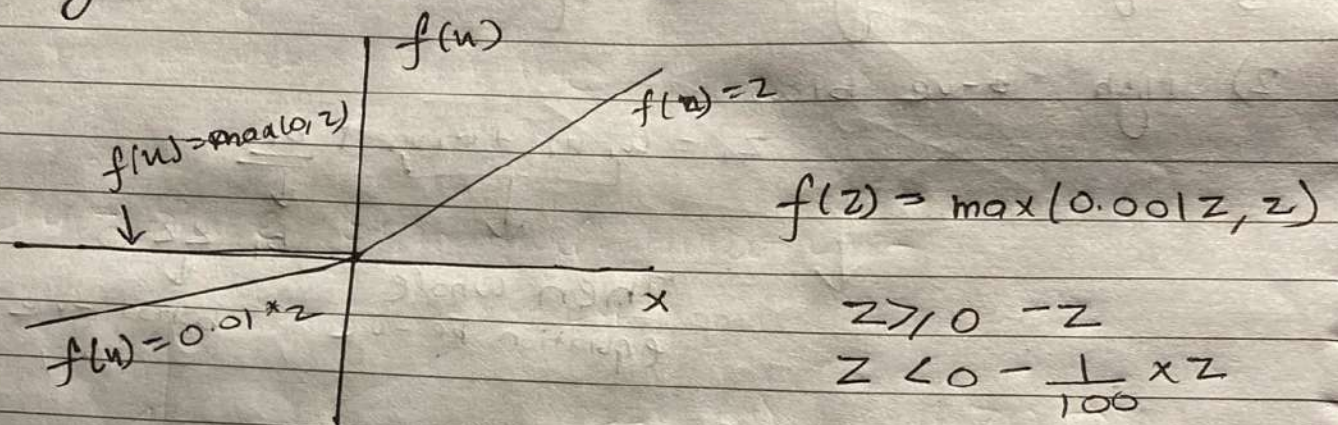


$$b_1 < 0$$

Solution 5

- Set low learning rates
- bias → +ve value → 0.01
- Don't use relu → use its variants

Leaky Relu



If $z = -ve$ output $\neq 0$ is $\boxed{0.01 \times z}$
derivative of z

$$f'(z) \begin{cases} z > 0 & -1 \\ z < 0 & 0.01 \end{cases}$$

Main problem of relu is it is solved derivative of relu is zero for $-ve$, its now $z' = \boxed{z \times 0.01}$

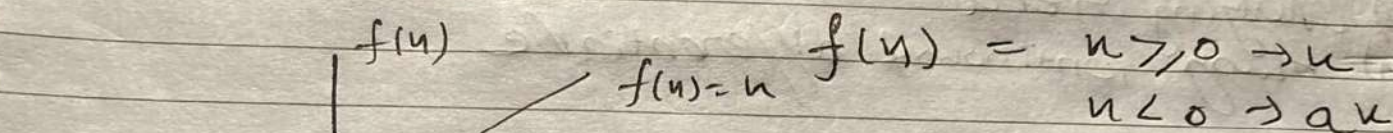
Advantages

- 1) Non Saturated → unbounded
- 2) Easy computation
- 3) not dying relu problem
- 4) close to zero centered -ve/+ve

Why 0.01

↳ Experimentally proved

Parametric Relu

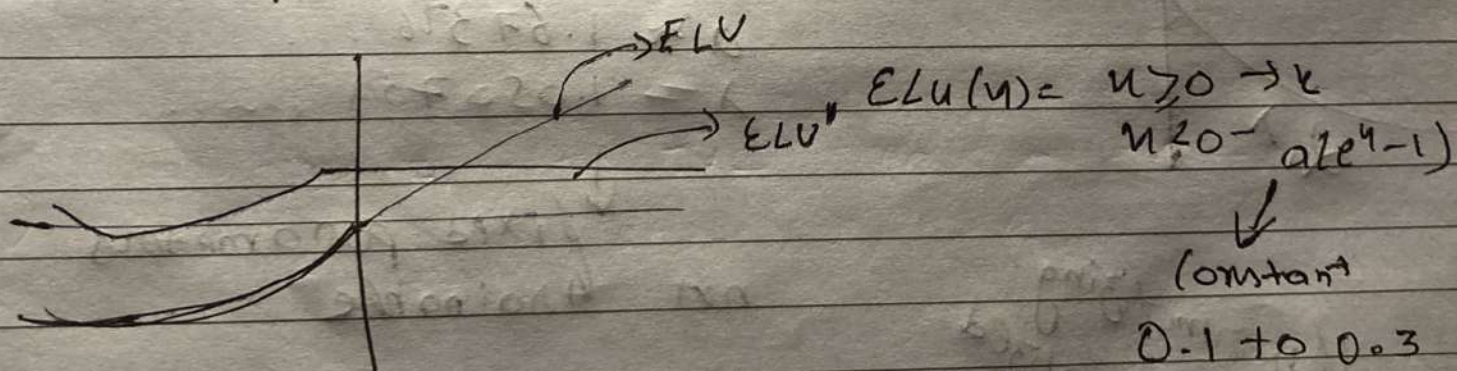


a = trainable parameter

↓
depending on data
value depends

flexible variables, sometimes
better than leaky ReLU

ELU → Exponential Linear Unit



Has performance better than ReLU,
function is always continuous

$$ELU'(u) = 1 \quad u > 0$$

$$\rightarrow ELU(u) \text{ if } u < 0$$

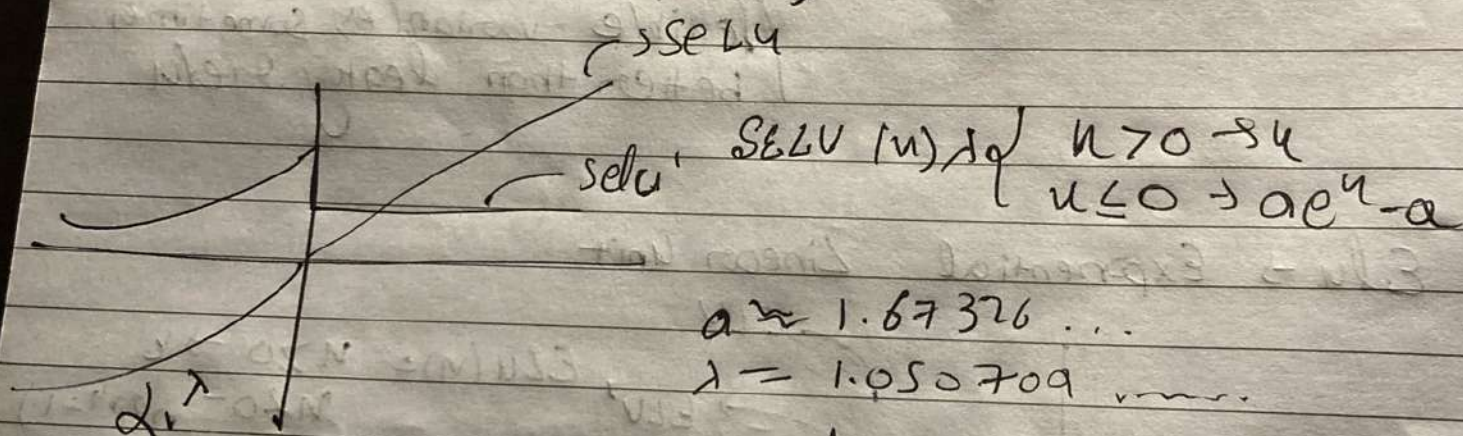
Advantage

- 1) Zero centered $\rightarrow$ convergence faster
- 2) generalized result are better (good on test data)
- 3) Unifying relu
- 4) Always continuous & differentiable

Dis Advantage

- 1) computationally expensive

Selu $\rightarrow$ Scale Exponential Linear unit



$\downarrow$ fixed parameters
not trainable

Self normalizing
 $\downarrow$
output is normalized
 $\downarrow$
mean 0, $\sigma = 1$

$\downarrow$
fast convergence

selu'(u)
$$\begin{cases} u > 0 \rightarrow 1 \\ u \leq 0 \rightarrow a e^{u-\lambda} \end{cases}$$